

Standardized Observable Validation API Specification

mdstats project

2026-07-30 (v3 result-identity and runtime closure)

Contents

1	Scope and ownership	1
2	Observable capability contract	1
3	Calls and recipes	2
4	Collection preflight	2
5	Parameter codecs	2
6	Analysis-owned result identity	2
7	Runtime evidence	3
8	Registered capabilities	3
9	Compatibility	3
10	Required tests	3

1 Scope and ownership

`mdstats.analysis.observable_validation` is an analysis-owned dispatch, preflight, and evidence facade. It does not implement RDF, coordination, topology, dynamics, spectra, diffusion, conductivity, or any other scientific observable. It invokes authoritative owner functions and preserves their native result objects.

The facade owns stable call IDs, parameter codecs, dependency-safe recipes, collection preflight, owner capability identity, runtime evidence, warnings, execution durations, and canonical identities of native results.

2 Observable capability contract

Each `ObservableCapability` records:

- stable `observable_id` and domain;
- owner module and function;
- stable owner-manual ID, source-tree path, and versioned documentation URI;
- machine-checkable `CollectionRequirement` values;
- dependency arguments and arguments that may only be supplied through native upstream-result bindings;
- required, supported, and one-of argument groups;
- parameter schema and codec identity;
- source-and-signature SHA-256 identity of the owner implementation;
- result type hint, implementation status, notes, and content digest.

A filesystem documentation path is not treated as an installed-wheel resource. The stable manual ID and URI remain valid in a wheel; the source path is an explicit source-package location hint. The wheel packages `mdstats/data/observable_owner_manuals.json` as the manual index.

3 Calls and recipes

`ObservableAnalysisCall` is immutable and JSON-safe. Nonfinite parameters, opaque objects, empty keys, and digest mismatch fail closed.

`ObservableAnalysisRecipe` validates at construction:

1. unique call IDs;
2. one common API version;
3. registered observable IDs;
4. supported arguments;
5. required and one-of argument groups;
6. binding-only native-result arguments;
7. required dependency bindings;
8. no self, unknown, cyclic, or forward dependencies.

Native results such as an `RDFResult` cannot be injected as JSON-shaped parameters. They must be supplied through `input_bindings` from a preceding call.

4 Collection preflight

Preflight validates scientific structure rather than mere attribute presence. Depending on the capability, it verifies:

- finite (`n_frames`, `n_atoms`, 3) positions;
- finite, nonsingular (`n_frames`, 3, 3) cells;
- positive one-dimensional atomic numbers;
- fixed population consistency;
- periodic-axis requirements;
- trajectory semantics;
- finite strictly increasing time axis;
- velocity shape and finiteness;
- finite stress or energy arrays where required.

Failure raises `ObservableRequirementError` before the owner function executes.

5 Parameter codecs

The facade currently provides versioned codecs for chemical pairs, cutoff registries, coordination filters, distance/hysteretic/reference connectivity, and JSON time intervals. Unsupported owner options remain absent from `supported_arguments` and fail closed rather than being passed with an incorrect type.

6 Analysis-owned result identity

Every successful call produces an `ObservableResultIdentity`. The identity contains:

- call and observable IDs;
- fully qualified native result type;
- serializer identity;
- canonical SHA-256 digest.

The canonicalizer traverses dataclasses, mappings, sequences, enums, scalars, and numeric/string NumPy arrays. Arrays are represented by dtype, shape, and content digest rather than copied into the invocation record. Object-

dtype arrays and unsupported opaque objects fail closed. Scientific result semantics and any full result-artifact schema remain owned by the authoritative analysis module.

7 Runtime evidence

`ObservableExecutionResult` preserves native results, result types, result identities, warnings, capability digests, and durations. Runtime identity records:

- observable API version;
- executing mdstats version from source code;
- separately reported installed-distribution version when available;
- source-tree or installed-package mode;
- executing module path and SHA-256;
- Python implementation/version, platform, machine;
- NumPy, SciPy, and ASE versions.

The executing source identity is authoritative when source code and an older installed wheel coexist.

8 Registered capabilities

The v3 facade retains the 22 implemented structural, topology, dynamics, spectral, diffusion, and conductivity IDs introduced in v2. New thermomechanical or energetic IDs are registered only after implementation under the thermomechanical and energetic validation architecture.

9 Compatibility

Readers accept v1/v2 call and recipe records and upgrade them to current semantics. Newly written capability records use schema v3. Legacy digests are not misrepresented as current digests.

10 Required tests

Tests shall cover:

- owner identity and implementation digests;
- dependency and binding-only rejection;
- preflight shape, finiteness, cell, time-axis, and velocity failures;
- codec decoding and unsupported options;
- all registered dependency chains;
- canonical result identities for all registered results;
- warning/runtime capture;
- source-tree and installed-wheel registry parity;
- packaged manual-index presence;
- unchanged native result types.